Relatório de Análise

Qualidade de Repositórios Java Populares do GitHub

Análise empírica — CBO, DIT e LCOM nos 1.000 repositórios Java mais estrelados

Disciplina	Experimentação de Software
Instituição	PUCMINAS
Período	01/2026
Autores	Augusto Fuscaldi Cerezo, Filipe Faria Melo

Índice

1. Introdução

- **1.1** Contextualização
- **1.2** Problema Foco do Experimento
- **1.3** Questões de Pesquisa
- **1.4** Definição de Métricas
- **1.5** Hipóteses
- **1.6** Objetivos

2. Metodologia

- **2.1** Passo a Passo do Experimento
- **2.2** Decisões Metodológicas
- **2.3** Materiais Utilizados
- **2.4** Métricas e suas Unidades

3. Visualização dos Resultados

- **3.1** Dados Coletados
- **3.2** Estatísticas Descritivas
- **3.3** Método de Análise dos Gráficos
- **3.4** RQ01 – Popularidade × Qualidade
- **3.5** RQ02 – Maturidade × Qualidade
- **3.6** RQ03 – Atividade × Qualidade
- **3.7** RQ04 – Tamanho × Qualidade

4. Discussão dos Resultados

- **4.1** RQ01 – Popularidade × Qualidade
- **4.2** RQ02 – Maturidade × Qualidade
- **4.3** RQ03 – Atividade × Qualidade
- **4.4** RQ04 – Tamanho × Qualidade

5. Conclusão

- **5.1** Tomada de Decisão
- **5.2** Sugestões Futuras

1. Introdução

1.1 Contextualização

O ecossistema Java é um dos mais consolidados no desenvolvimento de software, sendo amplamente utilizado em sistemas corporativos, aplicações Android, frameworks web e ferramentas de infraestrutura. O GitHub hospeda milhares de repositórios Java populares, muitos dos quais servem como referência de boas práticas para a comunidade. Analisar a qualidade interna desses sistemas, através de métricas de código orientado a objetos, pode revelar padrões relevantes sobre como popularidade, maturidade, atividade e tamanho se relacionam com características de qualidade do código.

1.2 Problema Foco do Experimento

Existe uma pergunta em aberto no contexto da Engenharia de Software empírica: projetos populares, maduros ou maiores necessariamente possuem melhor qualidade interna de código? Métricas como acoplamento (CBO), profundidade de herança (DIT) e coesão (LCOM) capturam aspectos estruturais do código que impactam manutenibilidade e testabilidade. Este estudo busca investigar se há relação mensurável entre essas métricas de qualidade e características do processo de desenvolvimento dos repositórios Java mais populares do GitHub.

1.3 Questões de Pesquisa

Código	Questão de Pesquisa
RQ01	Qual a relação entre a popularidade dos repositórios e as suas características de qualidade?
RQ02	Qual a relação entre a maturidade dos repositórios e as suas características de qualidade?
RQ03	Qual a relação entre a atividade dos repositórios e as suas características de qualidade?
RQ04	Qual a relação entre o tamanho dos repositórios e as suas características de qualidade?

1.4 Definição de Métricas

Métricas de Processo

Métrica	Proxy Utilizado
Popularidade	Número de estrelas (stars)
Maturidade	Idade em anos desde a criação do repositório
Atividade	Número de releases publicadas
Tamanho	Linhas de código Java (LOC) e linhas de comentários

Métricas de Qualidade (extraídas via ferramenta CK)

Métrica	Descrição

CBO	Coupling Between Objects — número de classes às quais uma classe está acoplada. Valores altos indicam forte dependência e menor facilidade de manutenção.
DIT	Depth of Inheritance Tree — profundidade da hierarquia de herança. Valores altos indicam maior reuso via herança, mas também maior complexidade.
LCOM	Lack of Cohesion of Methods — mede o quanto os métodos de uma classe não compartilham atributos internos. Valores altos indicam baixa coesão e possível necessidade de decomposição da classe.

Para cada repositório são calculadas a média e a mediana de cada métrica CK entre todas as classes analisadas. A mediana é preferível na análise por ser robusta a outliers.

1.5 Hipóteses

Hipótese	Enunciado
H1 (RQ01)	Repositórios mais populares tendem a ter maior CBO, pois projetos com muitos usuários costumam ser frameworks ou bibliotecas com alto grau de integração entre componentes.
H2 (RQ02)	Repositórios mais maduros tendem a ter menor LCOM, pois projetos com longa história de manutenção tendem a passar por refatorações que melhoram a coesão das classes.
H3 (RQ03)	Repositórios com mais releases tendem a ter menor CBO, pois ciclos de release frequentes podem incentivar boas práticas e redução de dependências.
H4 (RQ04)	Repositórios maiores (mais LOC) tendem a ter maior CBO e maior DIT, pois sistemas maiores naturalmente possuem mais componentes interdependentes e hierarquias de herança mais profundas.

1.6 Objetivos

Objetivo Principal: Investigar empiricamente a relação entre características de processo de desenvolvimento e métricas de qualidade de código nos 1.000 repositórios Java mais populares do GitHub.

Objetivos Específicos:

- Coletar dados de processo e produto para os 1.000 repositórios Java mais estrelados
- Extrair métricas CK (CBO, DIT, LCOM) via análise estática do código-fonte
- Calcular correlações de Pearson entre métricas de processo e qualidade
- Comparar médias e medianas para identificar influência de outliers
- Validar ou refutar as hipóteses formuladas com base em evidências empíricas

2. Metodologia

2.1 Passo a Passo do Experimento

Etapa 1 – Planejamento

- Definição das questões de pesquisa (RQ01–RQ04)
- Identificação das métricas de processo e qualidade
- Formulação de hipóteses

Etapa 2 – Coleta de Dados da API

- Autenticação na API GraphQL v4 do GitHub usando token pessoal
- Query dos repositórios com language:Java sort:stars-desc
- Paginação de 10 repositórios por requisição com cursor
- Repaginação automática por filtro stars:<min quando a paginação esgota
- Deduplicação por nome completo entre páginas
- Delay de 0,5s entre requisições para respeitar rate limiting

Etapa 3 – Extração de Métricas de Código

- Para cada repositório: sparse checkout (apenas arquivos .java)
- Contagem de LOC via ferramenta cloc (fallback para contagem manual)
- Análise CK: java -jar ck.jar {repo_dir} true 0 false {out_dir}
- Leitura dos arquivos CSV/JSON gerados pelo CK
- Cálculo de média e mediana de CBO, DIT, LCOM por repositório
- Remoção do clone local após extração
- Paralelização com ThreadPoolExecutor (4 workers simultâneos)

Etapa 4 – Processamento e Exportação

- Cálculo de métricas derivadas (idade em anos, dias desde push, percentual de issues)
- Exportação para CSV com todas as métricas de processo e qualidade

Etapa 5 – Análise

- Cálculo do coeficiente de correlação de Pearson (r) para cada par processo × qualidade
- Comparação de médias vs. medianas para identificar influência de outliers
- Geração de gráficos de dispersão com linha de tendência
- Validação das hipóteses com base nos resultados observados

2.2 Decisões Metodológicas

Decisão	Justificativa

Sparse Checkout	Em vez de clonar o repositório completo, utiliza-se sparse checkout com --filter=blob:none limitado a **/*.java (glob de arquivos Java). Isso reduz drasticamente o tempo de clone e o uso de disco, além de evitar erros de caminho longo no Windows.
Timeout do CK	Repositórios muito grandes (ex.: hazelcast, spring-boot) podem demorar mais de 5 minutos no CK. Foi definido timeout padrão de 120s, após o qual o repositório é ignorado — as colunas CK ficam vazias para esses casos.
Média vs. Mediana	Ambas são reportadas. A mediana é a medida principal por ser robusta a outliers (como repositórios com LCOM extremamente alto). A comparação entre as duas nos gráficos permite diagnosticar distorções causadas por valores extremos.
Correlação de Pearson	Escolhida por ser a métrica padrão para medir relações lineares entre variáveis contínuas. Valores próximos de zero indicam ausência de relação linear, sem excluir relações não-lineares.
Atividade = Releases	O número de releases foi escolhido como proxy de atividade por representar entregas formais e versionadas, diferente de pushedAt (pushes triviais) ou pr_count (contribuições externas).

2.3 Materiais Utilizados

Item	Detalhes
API	GitHub GraphQL API v4 — https://api.github.com/graphql Autenticação: Token pessoal com permissão public_repo
Ferramenta CK	ck-0.7.1-SNAPSHOT-jar-with-dependencies.jar
Linguagem	Python 3.12
Bibliotecas	requests, subprocess, concurrent.futures, tkinter, matplotlib, numpy
Infraestrutura	Windows 11 · Java no PATH (execução do CK) · Git no PATH (clonagem)

2.4 Métricas e suas Unidades

Métrica	Unidade	Fonte
Estrelas	Contagem	API GitHub
Idade	Anos (dias / 365)	Calculado a partir de createdAt
Releases	Contagem	API GitHub
LOC Java	Linhas de código	cloc / contagem manual
Linhas de Comentários	Linhas	cloc / contagem manual
CBO (média / mediana)	Adimensional	CK por repositório

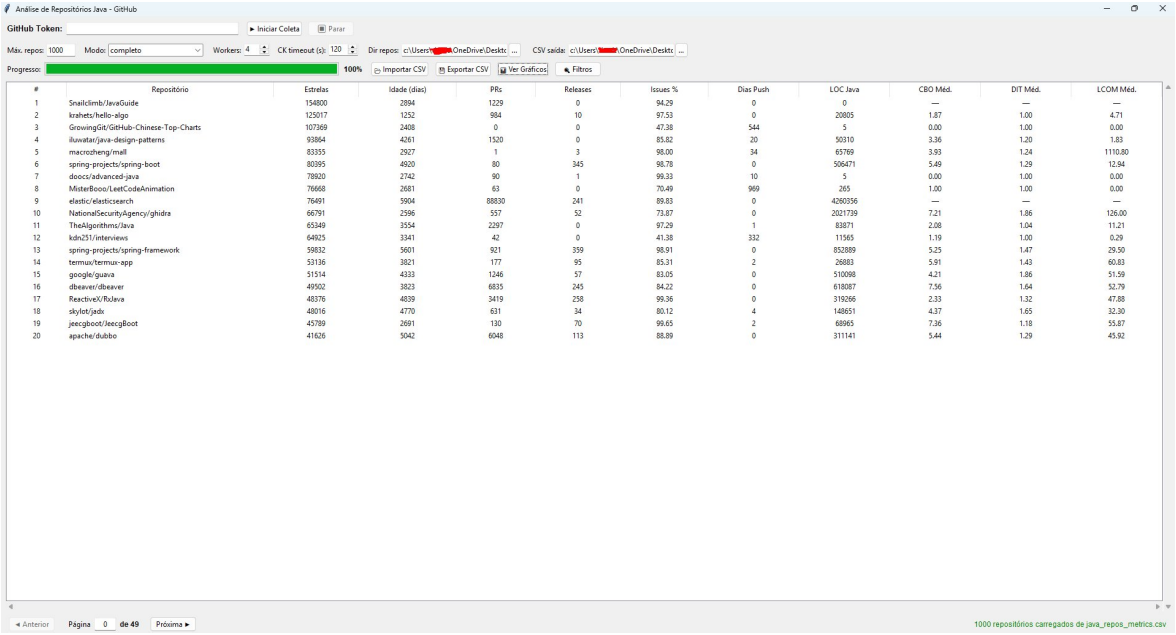
DIT (média / mediana)	Adimensional	CK por repositório
LCOM (média / mediana)	Adimensional	CK por repositório

3. Visualização dos Resultados

3.1 Dados Coletados

Descrição	Valor
Total de repositórios analisados	1.000
Repositórios com métricas CK válidas	971
Repositórios sem CK (timeout ou falha)	29

Interface da Ferramenta Desenvolvida



#	Repositório	Estrelas	Idade (dias)	PRs	Releases	Issues %	Dias Push	LOC Java	CBO Méd.	DIT Méd.	LCOM Méd.
1	Snailclimb/JavaGuide	154800	2894	1229	0	94.29	0	0	—	—	—
2	krahets/hello-algo	125017	1252	984	10	97.53	0	20805	1.87	1.00	4.71
3	GrowingGit/GitHub-Chinese-Top-Charts	107369	2408	0	0	47.38	544	5	0.00	1.00	0.00
4	iluwatar/java-design-patterns	93864	4261	1520	0	85.82	20	50310	3.36	1.20	1.83
5	macrozheng/mall	83355	2927	1	3	98.00	34	65769	3.93	1.24	1110.80
6	spring-projects/spring-boot	80395	4920	80	345	98.78	0	506471	5.49	1.29	12.94
7	doocs/advanced-java	78920	2742	90	1	99.33	10	5	0.00	1.00	0.00
8	MisterBooo/LeetCodeAnimation	76668	2681	63	0	70.49	969	265	1.00	1.00	0.00
9	elastic/elasticsearch	76491	5904	88830	241	89.83	0	4260356	—	—	—
10	NationalSecurityAgency/ghidra	66791	2596	557	52	73.87	0	2021739	7.21	1.86	126.00
11	TheAlgorithms/Java	65349	3554	2297	0	97.29	1	83871	2.08	1.04	11.21
12	kdn251/interviews	64925	3341	42	0	41.38	332	11565	1.19	1.00	0.29
13	spring-projects/spring-framework	59832	5601	921	359	98.91	0	852889	5.25	1.47	29.50
14	termux/termux-app	53136	3821	177	95	85.31	2	26883	5.91	1.43	60.83
15	google/guava	51514	4333	1246	57	83.05	0	510098	4.21	1.86	51.59
16	dbeaver/dbeaver	49502	3823	6835	245	84.22	0	618087	7.56	1.64	52.79
17	ReactiveX/RxJava	48376	4839	3419	258	99.36	0	319266	2.33	1.32	47.88
18	skylot/jadx	48016	4770	631	34	80.12	4	148651	4.37	1.65	32.30
19	jeecgboot/JeecgBoot	45789	2691	130	70	99.65	2	68965	7.36	1.18	55.87
20	apache/dubbo	41626	5042	6048	113	88.89	0	311141	5.44	1.29	45.92

Figura 1 – GUI principal: coleta e visualização dos dados

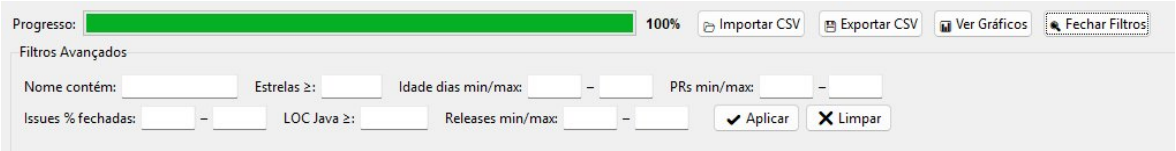


Figura 2 – Filtros avançados e controles de exportação

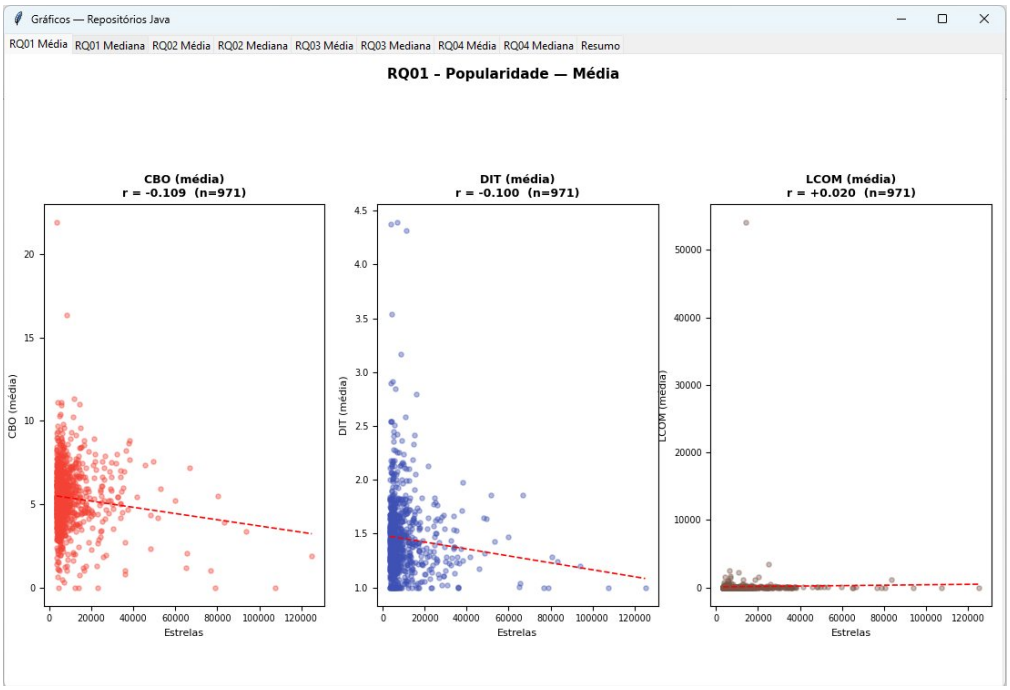


Figura 3 – Janela de gráficos com abas por RQ

3.2 Estatísticas Descritivas

Estatísticas Descritivas

Métrica	N	Mín	Mediana	Média	Máx
Estrelas	1000	3,473.00	5,902.50	9,808.95	154,800.00
Idade (anos)	1000	0.55	10.30	10.14	17.48
Releases	1000	0.00	11.00	41.31	1,000.00
LOC Java	999	0.00	18,460.00	150,259.74	10,752,652.00
Linhas Comentários	999	0.00	4,999.00	54,041.04	12,466,451.00
CBO Média	971	0.00	5.33	5.39	21.89
DIT Média	971	1.00	1.39	1.46	4.39
LCOM Média	971	0.00	24.56	122.05	54,025.11

Figura 4 – Estatísticas descritivas (saída da GUI)

Métrica	N	Mín	Mediana	Média	Máx
Estrelas	1000	3.473	5.902,50	9.808,95	154.800
Idade (anos)	1000	0,55	10,30	10,14	17,48
Releases	1000	0	11	41,31	1.000
LOC Java	999	0	18.460	150.259,74	10.752.652
Linhas Comentários	999	0	4.999	54.041,04	12.466.451
CBO Média	971	0	5,33	5,39	21,89
DIT Média	971	1,00	1,39	1,46	4,39
LCOM Média	971	0	24,56	122,05	54.025,11

> Destaque para a grande assimetria nas distribuições: Releases (mediana 11, média 41,31), LOC Java (mediana 18.460, média 150.259) e especialmente LCOM (mediana 24,56, média 122,05) revelam distribuições fortemente assimétricas à direita, reforçando a importância de usar a mediana como medida central na análise.

Top 20 Repositórios – Amostra dos Dados Coletados

#	Repositório	Estrelas	Idade (dias)	PRs	Releases	Issues %	Dias Push	LOC Java	CBO Méd.	DIT Méd.	LCOM Méd.
1	Snailclimb/JavaGuide	154800	2894	1229	0	94.29	0	0	—	—	—
2	krahets/hello-algo	125017	1252	984	10	97.53	0	20805	1.87	1.00	4.71
3	GrowingGit/GitHub-Chinese-Top-Charts	107369	2408	0	0	47.38	544	5	0.00	1.00	0.00
4	iluwatar/java-design-patterns	93864	4261	1520	0	85.82	20	50310	3.36	1.20	1.83
5	macrozheng/mall	83355	2927	1	3	98.00	34	65769	3.93	1.24	1110.80
6	spring-projects/spring-boot	80395	4920	80	345	98.78	0	506471	5.49	1.29	12.94
7	doocs/advanced-java	78920	2742	90	1	99.33	10	5	0.00	1.00	0.00
8	MisterBooo/LeetCodeAnimation	76668	2681	63	0	70.49	969	265	1.00	1.00	0.00
9	elastic/elasticsearch	76491	5904	88830	241	89.83	0	4260356	—	—	—
10	NationalSecurityAgency/ghidra	66791	2596	557	52	73.87	0	2021739	7.21	1.86	126.00
11	TheAlgorithms/Java	65349	3554	2297	0	97.29	1	83871	2.08	1.04	11.21
12	kdn251/interviews	64925	3341	42	0	41.38	332	11565	1.19	1.00	0.29
13	spring-projects/spring-framework	59832	5601	921	359	98.91	0	852889	5.25	1.47	29.50
14	termux/termux-app	53136	3821	177	95	85.31	2	26883	5.91	1.43	60.83
15	google/guava	51514	4333	1246	57	83.05	0	510098	4.21	1.86	51.59
16	dbeaver/dbeaver	49502	3823	6835	245	84.22	0	618087	7.56	1.64	52.79
17	ReactiveX/RxJava	48376	4839	3419	258	99.36	0	319266	2.33	1.32	47.88
18	skylot/jadx	48016	4770	631	34	80.12	4	148651	4.37	1.65	32.30
19	jeecgboot/JeecgBoot	45789	2691	130	70	99.65	2	68965	7.36	1.18	55.87
20	apache/dubbo	41626	5042	6048	113	88.89	0	311141	5.44	1.29	45.92

Figura 5 – Top 20 repositórios com todas as métricas coletadas

3.3 Método de Análise dos Gráficos

Todas as RQs são analisadas através de gráficos de dispersão que relacionam uma métrica de processo (eixo X) com uma métrica de qualidade (eixo Y). Cada ponto representa um repositório. Os elementos visuais presentes em cada gráfico são:

- Pontos coloridos — cada ponto é um repositório com ambas as métricas disponíveis
- Linha tracejada vermelha — regressão linear, indicando a tendência geral da relação
- r = valor — coeficiente de correlação de Pearson entre as duas variáveis
- n = valor — número de repositórios com dados válidos para aquele par

Coeficiente de Correlação de Pearson (r)

O coeficiente de Pearson mede o grau e a direção de uma relação linear entre duas variáveis. Seu valor varia sempre entre -1 e +1. Valores positivos: variáveis crescem juntas; negativos: uma cresce enquanto a outra diminui. *Correlação não implica causalidade.*

|r|	Classificação
0,00 – 0,19	Correlação desprezível
0,20 – 0,39	Correlação fraca
0,40 – 0,59	Correlação moderada
0,60 – 0,79	Correlação forte
0,80 – 1,00	Correlação muito forte

Média vs. Mediana

Cada RQ é apresentada em duas versões — usando a média e usando a mediana das métricas CK. A comparação permite diagnosticar a influência de outliers: se os coeficientes r forem similares, outliers têm

pouco impacto; se divergirem significativamente, a média está sendo distorcida por valores extremos e a mediana é a medida mais confiável.

3.4 RQ01 – Popularidade × Qualidade

Estrelas como proxy de popularidade. Eixo X: número de estrelas. Eixo Y: CBO / DIT / LCOM.

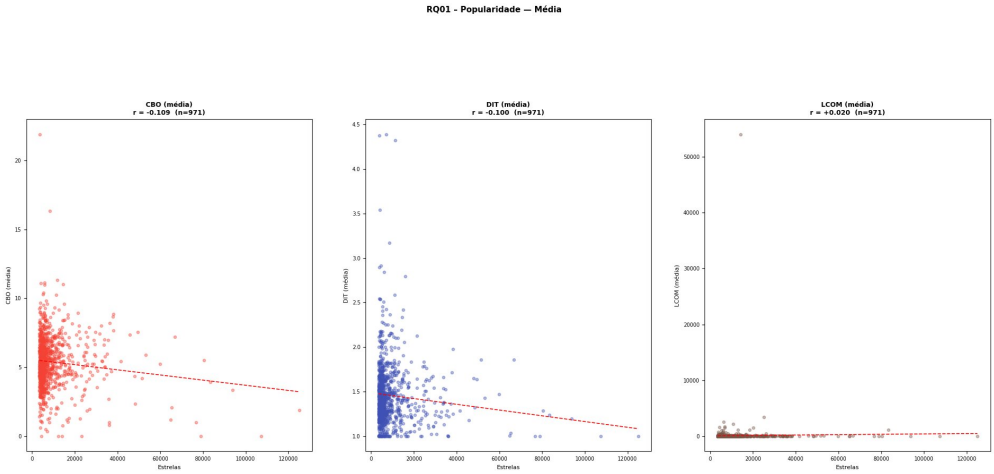


Figura 6 – RQ01: Popularidade × Qualidade (Média)

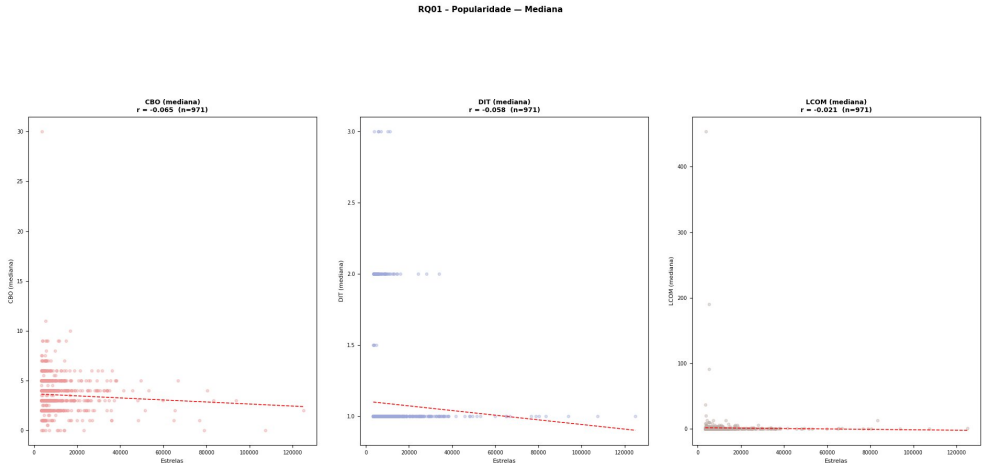


Figura 7 – RQ01: Popularidade × Qualidade (Mediana)

Par	r (usando média)	r (usando mediana)	n
Estrelas × CBO	-0,109	-0,065	971
Estrelas × DIT	-0,100	-0,058	971
Estrelas × LCOM	+0,020	-0,021	971

3.5 RQ02 – Maturidade × Qualidade

Idade em anos como proxy de maturidade. Eixo X: idade (anos). Eixo Y: CBO / DIT / LCOM.

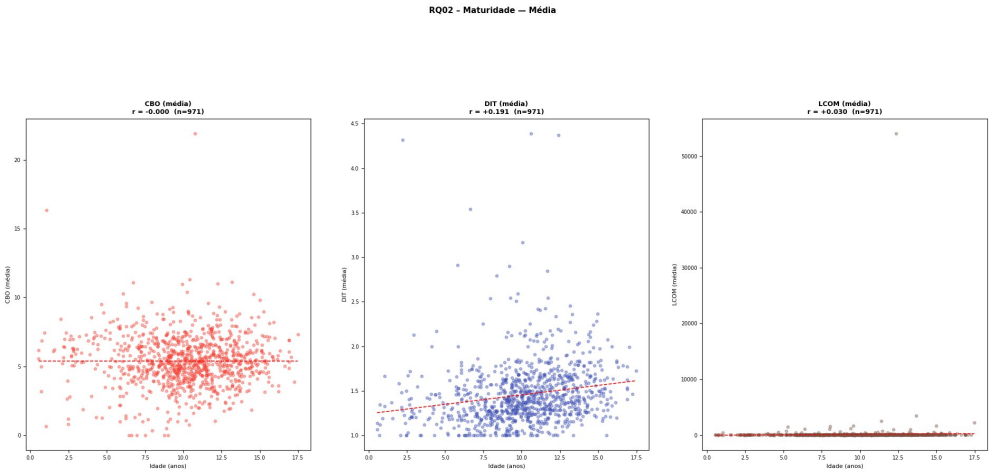


Figura 8 – RQ02: Maturidade × Qualidade (Média)

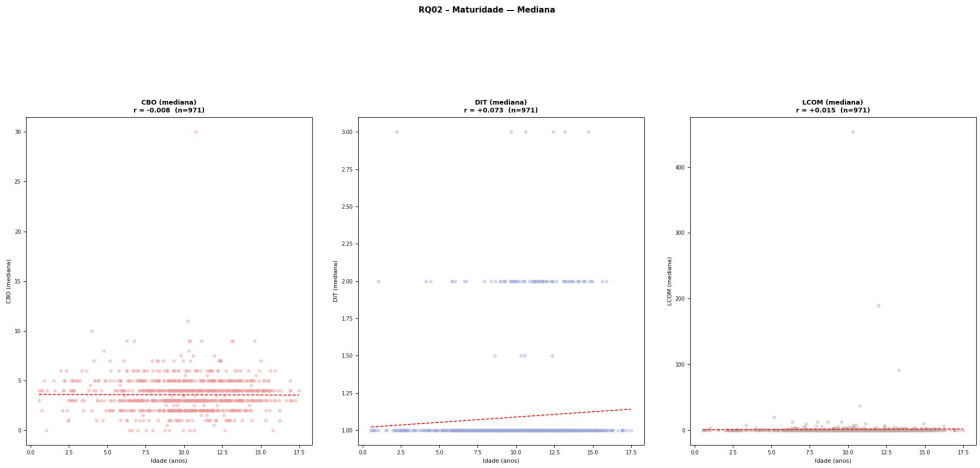


Figura 9 – RQ02: Maturidade × Qualidade (Mediana)

Par	r (usando média)	r (usando mediana)	n
Idade × CBO	0,000	-0,008	971
Idade × DIT	+0,191	+0,073	971
Idade × LCOM	+0,030	+0,015	971

3.6 RQ03 – Atividade × Qualidade

Número de releases como proxy de atividade. Eixo X: releases. Eixo Y: CBO / DIT / LCOM.

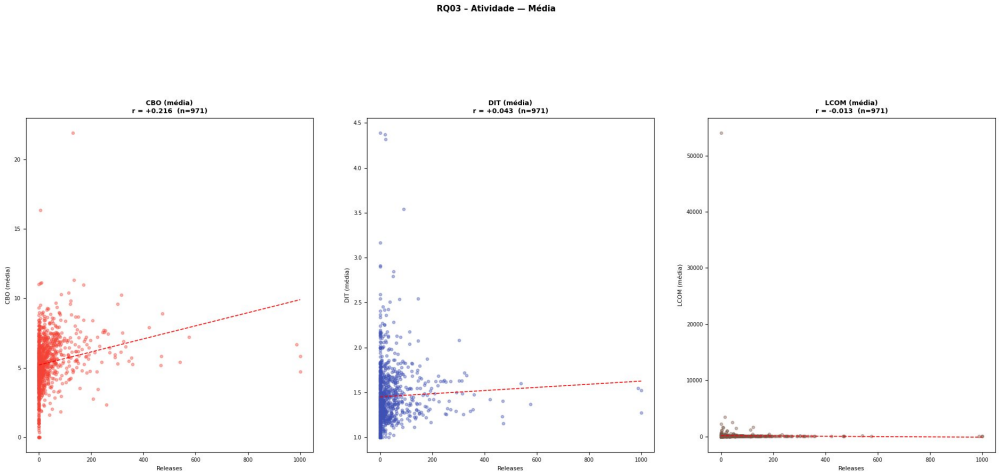


Figura 10 – RQ03: Atividade × Qualidade (Média)

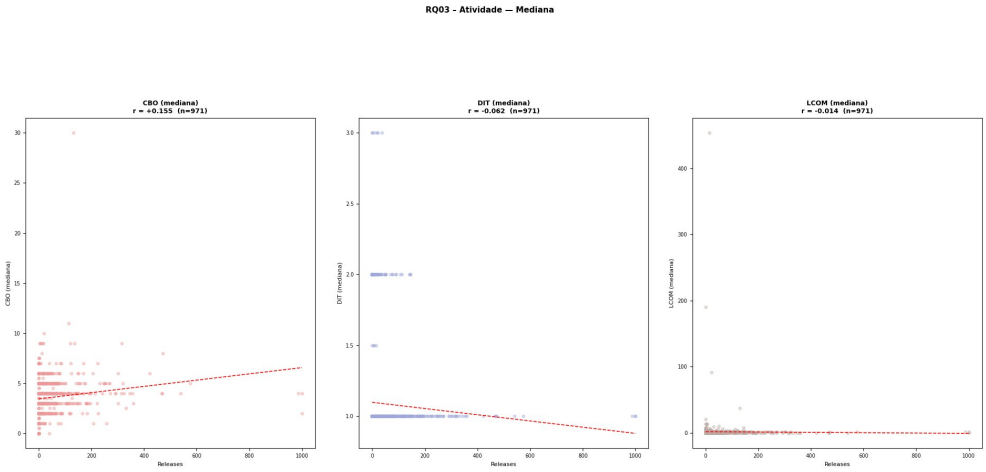


Figura 11 – RQ03: Atividade × Qualidade (Mediana)

Par	r (usando média)	r (usando mediana)	n
Releases × CBO	+0,216	+0,155	971
Releases × DIT	+0,043	-0,062	971
Releases × LCOM	-0,013	-0,014	971

3.7 RQ04 – Tamanho × Qualidade

LOC e linhas de comentários como proxies de tamanho. Eixo X: LOC / comentários. Eixo Y: CBO / DIT / LCOM.

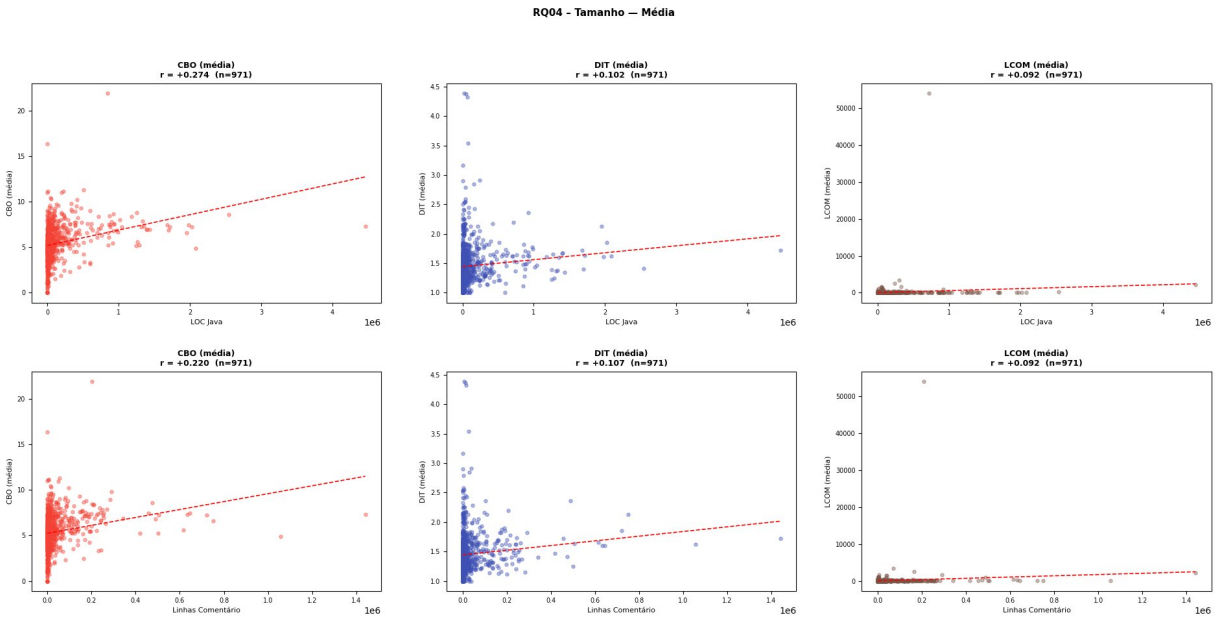


Figura 12 – RQ04: Tamanho × Qualidade (Média — LOC e Comentários)

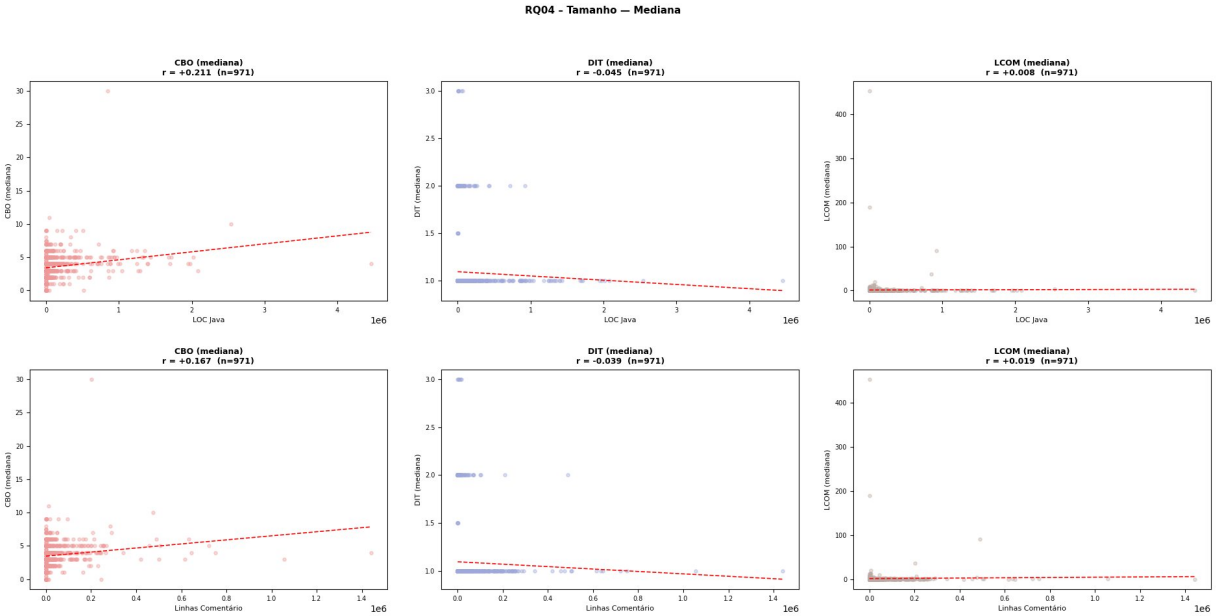


Figura 13 – RQ04: Tamanho × Qualidade (Mediana — LOC e Comentários)

Par	r (usando média)	r (usando mediana)	n
LOC × CBO	+0,274	+0,211	971
LOC × DIT	+0,102	-0,045	971
LOC × LCOM	+0,092	+0,008	971
Comentários × CBO	+0,220	+0,167	971
Comentários × DIT	+0,107	-0,039	971
Comentários × LCOM	+0,092	+0,019	971

Resumo Geral – Correlação de Pearson (processo × qualidade)

Correlação de Pearson (processo × qualidade)

Processo	vs CBO (r)	vs DIT (r)	vs LCOM (r)
Estrelas	-0.1086 (n=971)	-0.1000 (n=971)	+0.0201 (n=971)
Idade (anos)	-0.0001 (n=971)	+0.1906 (n=971)	+0.0301 (n=971)
Releases	+0.2161 (n=971)	+0.0430 (n=971)	-0.0130 (n=971)
LOC Java	+0.2739 (n=971)	+0.1021 (n=971)	+0.0923 (n=971)
Linhas Comentários	+0.2198 (n=971)	+0.1073 (n=971)	+0.0923 (n=971)

Figura 14 – Tabela completa de correlações de Pearson (saída da GUI)

Métrica de Processo	vs CBO (r)	vs DIT (r)	vs LCOM (r)
Estrelas	-0,1086	-0,1000	+0,0201
Idade (anos)	-0,0001	+0,1906	+0,0301
Releases	+0,2161	+0,0430	-0,0130
LOC Java	+0,2739	+0,1021	+0,0923
Linhas Comentários	+0,2198	+0,1073	+0,0923

4. Discussão dos Resultados

4.1 RQ01 – Popularidade × Qualidade

Hipótese	**Repositórios mais populares tendem a ter maior CBO por serem frameworks/bibliotecas com alto grau de integração entre componentes.**
Resultado Observado	As correlações entre número de estrelas e todas as métricas de qualidade foram desprezíveis (|r| < 0,11), com tendências ligeiramente negativas para CBO e DIT.
Veredito	■ Refutada

Par	r (média)	r (mediana)	Classificação
Estrelas × CBO	-0,109	-0,065	Desprezível
Estrelas × DIT	-0,100	-0,058	Desprezível
Estrelas × LCOM	+0,020	-0,021	Desprezível

Análise: A hipótese foi refutada. Esperava-se que repos populares, por serem tipicamente frameworks e bibliotecas amplamente integradas, apresentassem maior CBO. Contudo, os dados mostram o oposto: há leve tendência negativa entre popularidade e acoplamento, sugerindo que projetos mais estrelados podem ter classes mais simples e menos dependentes, possivelmente pela maturidade de suas arquiteturas. Os valores próximos de zero para LCOM (r=+0,020 e r=-0,021 com mediana) indicam ausência total de relação linear entre popularidade e coesão das classes.

4.2 RQ02 – Maturidade × Qualidade

Hipótese	**Repositórios mais maduros tendem a ter menor LCOM, pois projetos com longa história de manutenção passam por refatorações que melhoram a coesão.**
Resultado Observado	Correlações desprezíveis para CBO e LCOM. O DIT apresentou correlação positiva mais expressiva com a média (r=+0,191), mas cai para desprezível com a mediana (r=+0,073).
Veredito	■ Refutada

Par	r (média)	r (mediana)	Classificação
Idade × CBO	0,000	-0,008	Desprezível
Idade × DIT	+0,191	+0,073	Desprezível
Idade × LCOM	+0,030	+0,015	Desprezível

Análise: A hipótese foi refutada. Não há evidência de que repositórios mais maduros possuam menor LCOM — correlação de r=+0,030 (média) e +0,015 (mediana), praticamente nula. O achado mais interessante é o DIT: r=+0,191 com a média sugere que repositórios mais antigos tendem a ter hierarquias de herança ligeiramente mais profundas, possivelmente acumulando camadas ao longo do tempo. Porém, ao usar a mediana (r=+0,073), o efeito se reduz consideravelmente, indicando forte influência de outliers. Para CBO, a correlação é exatamente 0,000 — maturidade e acoplamento são métricas completamente independentes nos dados analisados.

4.3 RQ03 – Atividade × Qualidade

Hipótese	**Repositórios com mais releases tendem a ter menor CBO, pois ciclos frequentes incentivam boas práticas e redução de dependências.**
Resultado Observado	O CBO apresentou correlação fraca positiva com releases (r=+0,216 na média, r=+0,155 na mediana). DIT e LCOM mostraram correlações desprezíveis.
Veredito	■ Refutada

Par	r (média)	r (mediana)	Classificação
Releases × CBO	+0,216	+0,155	Fraca (média) / Desprezível (mediana)
Releases × DIT	+0,043	-0,062	Desprezível
Releases × LCOM	-0,013	-0,014	Desprezível

Análise: A hipótese foi refutada. Ao invés de menor CBO, repositórios mais ativos tendem a apresentar CBO levemente maior. A explicação mais plausível é que projetos com muitas releases são, em geral, sistemas maiores e mais complexos — frameworks ou bibliotecas amplamente utilizadas que naturalmente possuem mais integrações. A divergência de sinal no DIT (média +0,043, mediana -0,062) é indicativo claro de distorção por outliers. Para LCOM, correlação praticamente nula em ambas as versões, confirmando que a frequência de releases não tem relação com a coesão das classes.

4.4 RQ04 – Tamanho × Qualidade

Hipótese	**Repositórios maiores (mais LOC) tendem a ter maior CBO e maior DIT, pois sistemas maiores possuem mais componentes interdependentes e hierarquias mais profundas.**
Resultado Observado	O CBO apresentou a correlação mais forte de todo o estudo com LOC (r=+0,274, fraca positiva). DIT e LCOM mostraram correlações desprezíveis, com DIT invertendo o sinal ao usar a mediana — forte influência de outliers.
Veredito	■■ Parcialmente Confirmada (apenas para CBO)

Par	r (média)	r (mediana)	Classificação

LOC × CBO	+0,274	+0,211	Fraca
LOC × DIT	+0,102	-0,045	Desprezível
LOC × LCOM	+0,092	+0,008	Desprezível
Comentários × CBO	+0,220	+0,167	Fraca (média) / Desprezível (mediana)
Comentários × DIT	+0,107	-0,039	Desprezível
Comentários × LCOM	+0,092	+0,019	Desprezível

Análise: A hipótese foi parcialmente confirmada apenas para CBO. A relação entre LOC e CBO (r=+0,274) é a correlação mais forte observada em todo o estudo e se mantém robusta ao usar a mediana (r=+0,211) — o tamanho do repositório é o preditor mais confiável de acoplamento entre as variáveis investigadas. Isso faz sentido estruturalmente: sistemas maiores possuem mais classes, e mais classes aumentam as oportunidades e a necessidade de dependências entre componentes. Para DIT, ao usar a mediana o valor inverte para -0,045, revelando relação espúria dirigida por outliers. Para LCOM, correlação desprezível com a média e essencialmente nula com a mediana (+0,008) — tamanho do código não prediz a coesão das classes.

5. Conclusão

A análise dos 971 repositórios Java com métricas CK válidas revelou que a relação entre características de processo e qualidade de código é, em geral, **desprezível a fraca**. Os coeficientes de Pearson obtidos variaram entre r=0,000 e r=+0,274, indicando que nenhuma das métricas de processo investigadas — popularidade, maturidade, atividade ou tamanho — é um preditor forte da qualidade estrutural do código. O único relacionamento classificado como 'fraco' é entre LOC e CBO (r=+0,274), e mesmo esse resultado explica menos de 8% da variância observada no acoplamento.

Achados por Questão de Pesquisa

RQ01 Média | RQ01 Mediana | RQ02 Média | RQ02 Mediana | RQ03 Média | RQ03 Mediana | RQ04 Média | RQ04 Mediana | Resumo

Estatísticas Descritivas

Métrica	N	Mín	Mediana	Média	Máx
Estrelas	1000	3,473.00	5,902.50	9,808.95	154,800.00
Idade (anos)	1000	0.55	10.30	10.14	17.48
Releases	1000	0.00	11.00	41.31	1,000.00
LOC Java	999	0.00	18,460.00	150,259.74	10,752,652.00
Linhas Comentários	999	0.00	4,999.00	54,041.04	12,466,451.00
CBO Média	971	0.00	5.33	5.39	21.89
DIT Média	971	1.00	1.39	1.46	4.39
LCOM Média	971	0.00	24.56	122.05	54,025.11

Correlação de Pearson (processo × qualidade)

Processo	vs CBO (r)	vs DIT (r)	vs LCOM (r)
Estrelas	-0.1086 (n=971)	-0.1000 (n=971)	+0.0201 (n=971)
Idade (anos)	-0.0001 (n=971)	+0.1906 (n=971)	+0.0301 (n=971)
Releases	+0.2161 (n=971)	+0.0430 (n=971)	-0.0130 (n=971)
LOC Java	+0.2739 (n=971)	+0.1021 (n=971)	+0.0923 (n=971)
Linhas Comentários	+0.2198 (n=971)	+0.1073 (n=971)	+0.0923 (n=971)

Figura 15 – Resumo: estatísticas descritivas e correlações de Pearson (saída da GUI)

RQ	Métrica de Processo	Qualidade mais correlacionada	r (mediana)	Hipótese
RQ01	Estrelas	CBO	-0,065	Refutada
RQ02	Idade (anos)	DIT	+0,073	Refutada
RQ03	Releases	CBO	+0,155	Refutada
RQ04	LOC Java	CBO	+0,211	Parcialmente Confirmada

Impacto dos Outliers

- O **LCOM** apresentou a maior distorção: média de 122,05 vs. mediana de 24,56 — razão de quase 5:1 — dominada por poucos repositórios com coesão extremamente baixa (máx. 54.025). Qualquer correlação calculada com a média de LCOM é, por isso, pouco confiável.
- O **DIT** também sofre influência relevante de outliers: em RQ02, a correlação com a média é +0,191, mas cai para +0,073 com a mediana. Em RQ04, o coeficiente inverte de sinal (+0,102 vs. -0,045), ilustrando por que a mediana foi adotada como medida principal.
- O **CBO** mostrou-se a métrica mais estável: correlações calculadas com média e mediana foram sempre na mesma direção e com magnitudes próximas, conferindo mais confiança aos resultados envolvendo essa métrica.

Resumo: Os resultados contradizem a intuição de que repositórios mais populares, maduros, ativos ou maiores necessariamente possuem melhor qualidade estrutural de código. O achado de maior significância prática é que repositórios maiores tendem a ter maior acoplamento entre objetos (LOC × CBO, r=+0,274), o que é esperado pela complexidade inerente a sistemas grandes, mas ainda assim classifica-se apenas como correlação fraca. Para os 1.000 repositórios Java mais populares do GitHub, as métricas de processo analisadas não são preditoras confiáveis da qualidade estrutural do código.

Limitações do Estudo

• Timeout do CK: Repositórios muito grandes tiveram o CK interrompido, gerando ausência de métricas de qualidade para 29 repos — justamente os maiores, introduzindo viés de seleção.

• Correlação linear: O coeficiente de Pearson mede apenas relação linear. Relações não-lineares entre tamanho e qualidade não seriam capturadas.

• Agregação por repositório: CBO, DIT e LCOM são calculados por classe e depois agregados (média/mediana) — essa agregação perde informação sobre a distribuição interna de qualidade.

• Apenas arquivos .java: O sparse checkout exclui outras linguagens, podendo subestimar LOC em projetos multilinguagem.

• Snapshot temporal: Os dados representam um único momento no tempo. A qualidade de código pode variar significativamente ao longo do ciclo de vida do projeto.

• Causalidade: Correlações encontradas não implicam causalidade. Um CBO elevado em repos populares pode ser consequência da popularidade ou do tipo de projeto (frameworks naturalmente mais acoplados).

5.1 Tomada de Decisão

• Popularidade: número de estrelas como proxy

• Maturidade: idade em anos (calculada a partir de createdAt)

• Atividade: número de releases (entregas formais versionadas)

• Tamanho: LOC Java e linhas de comentários via cloc

• Qualidade: métricas CK (CBO, DIT, LCOM) — média e mediana por repositório

• Correlação: coeficiente de Pearson com comparação média vs. mediana

• Clone: sparse checkout apenas de .java para eficiência e compatibilidade Windows

• Timeout CK: 120s por repositório, ignorado em caso de estouro

• Paralelismo: 4 workers simultâneos para clone + CK

• Ferramenta: Python 3.12 + CK 0.7.1 + GUI tkinter/matplotlib

---

5.2 Sugestões Futuras

Dados Adicionais

• Coletar número de contribuidores únicos por repositório como métrica adicional de atividade

• Incluir cobertura de testes (via JaCoCo ou similar) como métrica complementar de qualidade

• Analisar complexidade ciclomática (WMC) disponível no CK, além de CBO/DIT/LCOM

Novos Indicadores

- Analisar evolução temporal das métricas CK ao longo do histórico de commits
- Comparar qualidade entre repositórios de diferentes domínios (frameworks, aplicações, bibliotecas)
- Incluir métricas de processo mais granulares, como frequência de commits por período

Metodologia

- Aumentar o timeout do CK para repositórios acima de determinado limiar de LOC, evitando perda de dados para os maiores sistemas
- Realizar coleta longitudinal (snapshots periódicos) para capturar evolução da qualidade
- Estratificar análise por domínio de aplicação (ex.: frameworks vs. ferramentas vs. aplicações)